CHAINSWE: Benchmarking Coding Agents on Multi-Bug Software Maintenance

Qirui Jin[1‡], Lingching Tung[4], Kenan Li[2], Qiyang Shi[3], Yushi She[1], Huanzhong Jia[1], Harrison Zhao[5], Kejing Xia[1], Zhenbang Du[1], Jiaxin Pei[6], Zhenyu Zhang[6], Zhen Qi[7], Yuyan Duan[1], Wenke Lee[1], Zijian Jin[3‡]

[1]Georgia Institute of Technology [2]University College London [3]New York University
[4]NVIDIA [5]Cornell University [6]Stanford University [7]Northeastern University

[‡]**Correspondence:** qiruijin@gatech.edu, zj2076@nyu.edu

arXiv:2607.02606v2 [cs.SE] 1 Sep 2026

Abstract

Language model (LM) agents are increasingly deployed to maintain codebases over extended periods, fixing streams of related defects while carrying context from one fix to the next. Yet existing software engineering (SWE) benchmarks evaluate models one bug at a time: the repository is reset, the codebase is re-read, and a single self-contained issue is graded in isolation. This setting collapses a continuous maintenance workflow into a series of independent sessions, ignoring the cumulative dependencies that make real-world bug fixing challenging. To bridge this gap, we introduce CHAINSWE, the first benchmark for evaluating agents on sequential, dependent bug fixes within a shared codebase. We collect chronological chains of 304 issues across 54 Python projects, mined from six SWE-bench-family datasets. Our evaluation across a range of agents and models reveals a consistent performance drop by **up to 70%** as the chain length increases. Our data and code are available at C-lister/ChainSWE.

1 Introduction

Language model (LM) agents are now deployed across a wide range of software engineering tasks, including competitive programming (Quan et al., 2025; Jain et al., 2024), code repair (Xia et al., 2024; Li et al., 2026a), automatic environment setup (Hu et al., 2026; Li et al., 2026b), and full-repository construction (Ding et al., 2026; Yang et al., 2026). In particular, agent performance has improved rapidly on a family of issue-resolution benchmarks, including SWE-bench (Jimenez et al., 2024), SWE-bench Live (Zhang et al., 2025), SWE-rebench (Badertdinov et al., 2025) and SWE-rebench-v2 (Badertdinov et al., 2026), SWE-Gym (Pan et al., 2025), and SWE-bench Pro (Deng et al., 2025). Strong agents now successfully resolve a substantial fraction of issues on these live leaderboards (Yang et al., 2024; Wang et al., 2025).

However, every benchmark in this family follows the same basic evaluation protocol: a model is evaluated on a single, self-contained issue within a sandbox container. The agent starts from a clean checkout at a pre-specified runnable base commit, receives a single problem statement, edits the codebase, and is evaluated against pre-written tests harvested from that issue. This protocol measures whether a patch satisfies a local hard-coded test suite. However, it does not capture whether the agent avoids unnecessary edits, resolves latent multi-file dependencies, or manages context across an evolving sequence of related fixes. As a result, this per-issue schema misses **two** failure modes that only emerge when bugs are evaluated sequentially, repository state carries across rollouts, and available tests fail to capture the damage left by earlier patches:

- *Overshoot* (Figure 1 (a)), where an agent fixing an earlier bug modifies files beyond the gold-patch-intended area; when a downstream bug subsequently targets those same files, its edits are applied on a polluted baseline, and otherwise correct changes no longer produce the expected behavior. The `amaranth` chain in Figure 1(a) provides a representative example. The gold patch modifies four files, whereas the agent additionally rewrites three unrelated backend files. A subsequent issue targets two of these files, causing all eight downstream tests to fail.

- *Undershoot* (Figure 1 (b)), where the prior bug's gold patch is a multi-file refactor, but the agent ships only the subset named in the issue title or description; downstream tests then fail not because the current bug is mishandled but because they exercise code paths whose preconditions were never established. The `MONAI` chain in Figure 1(b) exhibits the opposite failure mode. Although the gold patch spans five files, the agent

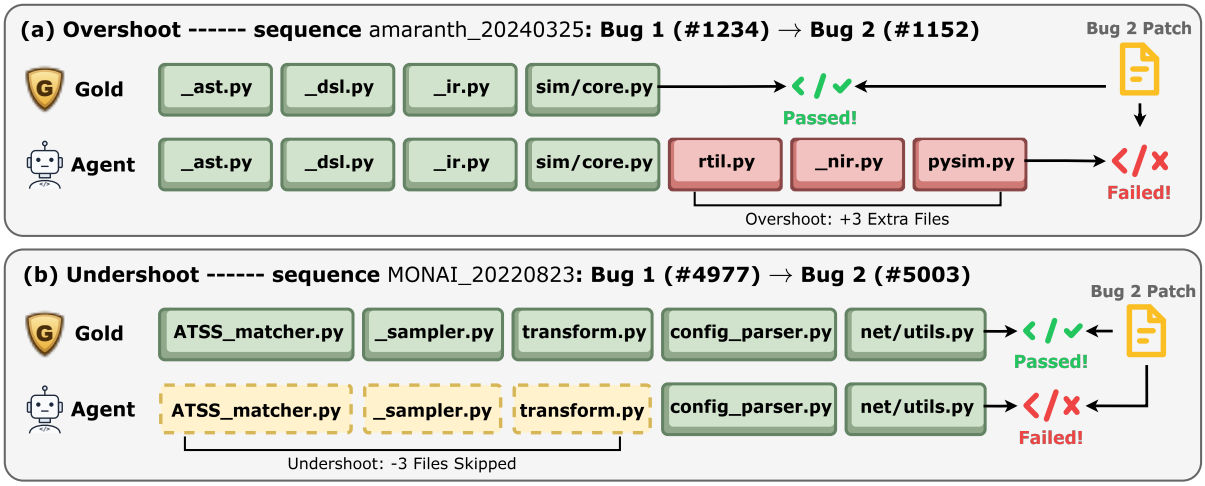


Figure 1: Two failure modes (*overshoot* and *undershoot*) exposed by ChainSWE rollouts.

edits only two files explicitly referenced in the issue description and omits three supporting refactors. As a result, a downstream test fails despite the agent producing the correct modification to the target file.

These examples show a mismatch between current benchmarks and the real-world software maintenance process. In practice, software engineers must leave the codebase in a state that is both correct for the current issue and usable as the starting point for future work. They also need to read through the surrounding context and adapt their fixes to the evolving repository state they inherit.

To bridge this gap, we introduce ChainSWE, a benchmark for *continuous* SWE rollouts. Each ChainSWE instance consists of a time-ordered chain of real bug fixes with overlapping modified files, functions, or classes from the same repository. We validate each chain by replaying the accumulated gold patches and associated test suites. During evaluation, at step k, the agent receives only the k-th issue statement and operates on the repository state produced by the previous steps. The agent is evaluated not only on whether it can solve each issue in isolation, but also on whether its earlier patches create a valid substrate for later fixes. We refer to failures caused by accumulated agent-generated state, rather than by the intrinsic difficulty of the current bug, as chain errors. Our ChainSWE dataset contains 100 bug chains spanning 304 instances and 54 repositories, drawn from six open-source SWE-bench-family datasets.

We evaluate seven state-of-the-art language models using a fixed agent scaffold (SWE-Edit (Zhang et al., 2026)) under three context-management configurations, where continuous rollout reveals failure modes that are not exposed under conventional isolated protocols. We observe that performance drops by up to 70% compared to the corresponding single-issue setting, with the sharpest declines at the deepest positions in a chain. Our results suggest that further progress in SWE agents will require advances in dependency tracking, long-horizon reasoning, and robust repository-state management, rather than improvements in isolated issue resolution alone.

The contributions of this paper are threefold:

- We introduce ChainSWE, the first open-source SWE benchmark that explicitly evaluates language models on sequential, multi-bug rollouts over real-world repositories.
- We develop a scalable pipeline for constructing and validating chained SWE tasks from existing benchmarks, enabling continuous evaluation over arbitrarily long bug-fix sequences.
- We conduct comprehensive evaluations of seven state-of-the-art LMs under multiple context-management configurations. We provide detailed analysis of their failure modes, offering guidance for future work on dependency-aware reasoning in code LMs and repository state management in agent harnesses.

2 Related Work

Repository-level issue resolution benchmarks. SWE-bench (Jimenez et al., 2024) introduced the

standard setup of pairing real GitHub issues with hidden test suites to grade repository-level coding agents, and a number of variants have followed. SWE-bench Verified re-audits and re-labels the original instances to remove under-specified or untestable tasks. SWE-bench Live (Zhang et al., 2025), SWE-rebench (Badertdinov et al., 2025) and SWE-rebench-v2 (Badertdinov et al., 2026) propose automated pipelines that continuously curate up-to-date issues to mitigate training-set contamination, while SWE-Gym (Pan et al., 2025) targets training rather than evaluation and SWE-bench Pro (Deng et al., 2025) extends both the difficulty and the size of the task set. BeyondSWE (Chen et al., 2026) broadens code-agent evaluation beyond single-repository bug fixing to cross-repository, domain specific, dependency migration, and document-to-repository tasks. ScaleSWE (Zhao et al., 2026) instead presents an automated pipeline for constructing large-scale, verified software engineering data for coding agent training.

Despite their differences, all of these benchmarks follow the same evaluation protocol: one issue per rollout, a fresh container reset to a clean base commit, and a fresh conversation, so each task is graded in complete isolation from every other. This design measures *single-shot* issue-resolution ability but deliberately removes the cross-task dependencies that characterize real maintenance, where a developer fixes a stream of related defects on top of a repository they themselves have just modified. Unlike these benchmarks, CHAINSWE asks the agent to work through a chain of related issues in a single rollout, without resetting between bugs, and grades both per-bug and full-chain success to observe failures caused by accumulated, self-generated state.

Agent harnesses for code. A coding *harness* connects a language model to tools—such as shell execution, file navigation, and code editing—and controls the context the model sees at each step. Existing harnesses take different approaches to context management: SWE-AGENT (Yang et al., 2024) uses a structured interface; OPENHANDS (Wang et al., 2025) automatically summarizes conversation history at context limit; CLAUDE CODE (Anthropic, 2025) further proposes sub-agent delegation and task planning for better long-context management; and AGENTLESS (Xia et al., 2024) replaces an autonomous loop with a fixed, step-by-step pipeline. Related work also considers budget-aware observation planning and the reliability of self-improving agent workflows (Peng et al., 2026; Wang et al., 2026a; Liu et al., 2026; Wang et al., 2026b). Most relevant to our study is SWE-EDIT (Zhang et al., 2026), which uses a constrained edit format and an optional sub-agent to apply code changes, lowering token costs during long interactions. These harnesses primarily differ in how they manage state across multiple turns. We adopt SWE-EDIT as a baseline framework to test three distinct context-management strategies: full history, summarization, and sub-agent editing. Using CHAINSWE, we measure how each strategy performs as conversation history and code edits accumulate across a continuous, multi-bug session.

Sequential and continual reasoning benchmarks. Another line of work (Liu et al., 2024; Ma et al., 2024) evaluates language agents in multi-step environments where success requires planning, tool use, and state tracking. Recent memory benchmarks further test whether assistants can retain and update information over long conversations or multi-session user histories (Maharana et al., 2024; Wu et al., 2025). These settings are sequential, but they typically reset between tasks or evaluate memory over dialogue state rather than over persistent changes to an external artifact. To the best of our knowledge, CHAINSWE is the first benchmark that combines real, test-validated repository bug fixes, multi-bug chains constructed with explicit code-overlap, and an evaluation protocol that distinguishes per-bug from full-chain success.

3 CHAINSWE

This section illustrates the data collection pipeline of CHAINSWE in detail.

3.1 Task Features

CHAINSWE pools instances from six existing repository-level SWE benchmarks: SWE-bench (Jimenez et al., 2024), SWE-bench Live (Zhang et al., 2025), SWE-rebench (Badertdinov et al., 2025) and SWE-rebench-v2 (Badertdinov et al., 2026), SWE-Gym (Pan et al., 2025), and SWE-bench Pro (Deng et al., 2025). Every instance is a tuple of repository, base commit, problem statement, gold patch, test patch, `FAIL_TO_PASS` test list, `PASS_TO_PASS` test list, and test commands. All six benchmarks target Python projects and ship pre-built Docker images for reproducible test exe-

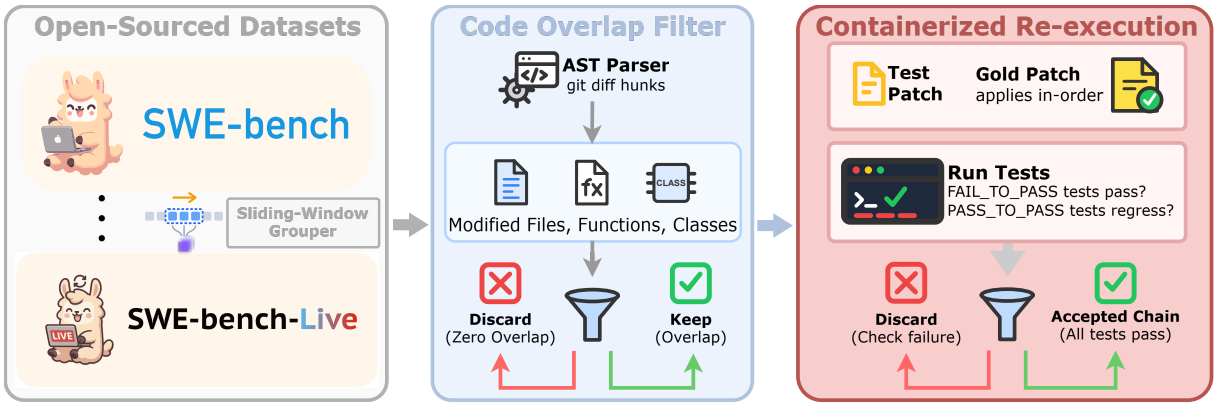


Figure 2: The data mining pipeline for ChainSWE.

cution, which enables evaluation of an arbitrary subset under a single harness. An instance of ChainSWE is a unified list of such instances, each ordered chronologically with the Docker image reset to the base commit of the first bug in the chain, tagged with a unique chain ID.

3.2 Mining Chains

As illustrated in Figure 2, we synthesize ChainSWE chains in two stages: code-overlap scoring and containerized test re-execution. For each repository in the pooled source, we collect all of its instances and sort them by commit date. We then group consecutive instances into chains using a sliding window, joining two instances into the same chain if either of the following conditions holds:

1. The latter's base commit is the immediate successor of the former's (i.e., no intervening commits in the repository's history).

2. The latter's base commit is not the immediate successor of the former's, but its gold patch and test patch apply cleanly on top of the former's resolved state.

The resulting candidate chains are passed to the filtering stages below; note that a chain of length l also induces all contiguous sub-chains of lengths ranging from 2 to l.

Code-overlap heuristic filter. For each gold patch, we use an AST parser on the `git diff` hunks to extract the modified files, functions, and classes, and discard any window whose bugs have zero pairwise overlap.

Test re-execution filter. The heuristic above is a cheap signal but does not guarantee that the chain actually builds. We therefore run a containerized filter that, for each candidate chain, pulls the source benchmark's Docker image, starts a fresh container at the chain's base commit, and applies—in order—each bug's test patch followed by its gold patch (the same setting as Oracle; see Section 4.3). Then, we run the bug's test commands and verify that all `FAIL_TO_PASS` tests now pass and no `PASS_TO_PASS` tests regress. A chain is accepted only if all filter stages clear.

3.3 Data Selection

After running the pipeline above, we obtained candidate chains with lengths ranging from 2 to 5. During data selection, we intentionally drop all chains of length 2 for two reasons:

1. Length-2 chains are too short to constitute a meaningful stress test: with only one prior bug, there is no meaningful accumulation of context or compounding of patch errors.

2. Most length-2 sequences are already embedded as contiguous prefixes inside length-3 or longer chains, so evaluating them separately would be largely redundant.

For the remaining chains, we apply a second round of difficulty-based selection. Because ChainSWE targets the cross-task state-management capabilities of agent harnesses rather than raw bug-solving ability, each chain should be *individually tractable* once state propagation is removed: a chain that fails because a single bug is intrinsically too hard tells us little about how well an agent carries state across fixes. We therefore pre-run Oracle mode (see Section 4.3) on BaseLinewith `Qwen3.7-Max` and retain only chains for which more than half of the bugs are solved (e.g., for a chain of length k, at least $\lfloor k/2 \rfloor + 1$ bugs must pass). This filters out chains whose difficulty

is dominated by the per-bug task rather than by accumulated repository state.

3.4 Dataset Statistics

The final version of ChainSWE yields 100 accepted chains with an average length of 3.04, comprising 304 bug-fix tasks drawn from 54 unique repositories and spanning April 2019 to August 2025. A detailed breakdown is provided in the Appendix A.

4 Evaluation

4.1 Agent Configurations

We hold the agent scaffold fixed across all experiments and vary only how it manages its context, so that performance differences can be attributed to the model and the context-management strategy rather than to the harness. We use SWE-Edit (Zhang et al., 2026), a variant of SWE-agent; we evaluate three configurations that differ only in how the trajectory is kept within the context window as it grows:

Baseline. The base SWE-Edit agent with no augmentations, identical to the original SWE-agent. It retains the full interaction transcript in context and applies edits with an exact string-replacement editor; no context compaction of any kind is performed.

Summarize. Baseline augmented with conversation summarization. Once the running context exceeds a fixed budget (50K input tokens), we replace the earlier portion of the history with a model-written structured summary while retaining the 50 most recent messages.

Sub-Agent. Sub-Agent offloads file viewing and editing to dedicated viewer and editor subagents, rather than placing file contents and diffs directly in the main agent's context. The main agent issues a natural-language instruction describing the intended change, and a separate, typically smaller, editor model materializes the concrete patch. This keeps the most token-heavy content out of the primary trajectory; no summarization is applied.

All three configurations share the same model, tools, and prompts (see Appendix D).

4.2 Language Models

We evaluate seven LMs spanning different long-context management capacities and tool-use strengths: `GPT-5.5`, `GPT-5.4-mini`, `GPT-5.4-nano`, `Claude-Opus-4.7`, `Claude-Opus-4.5`, `DeepSeek-V4-Pro`, `Gemini-3.1-Pro`. All experiments use the API's default inference parameters. For reasoning-capable models, we set thinking effort to `medium` to balance performance and cost. Table 1 summarizes the models used in this paper.

Model	SWE-bench Pro Score	Context
GPT-5.5	58.6%	1M
GPT-5.4-mini	54.4%	400K
GPT-5.4-nano	52.4%	400K
Claude-Opus-4.7	64.3%	1M
Claude-Opus-4.5	57.1%	200K
DeepSeek-V4-Pro	54.4%	1M
Gemini-3.1-Pro	54.2%	1M

Table 1: Language models evaluated on ChainSWE. Exact API context limits are those exposed by the serving provider at evaluation time.

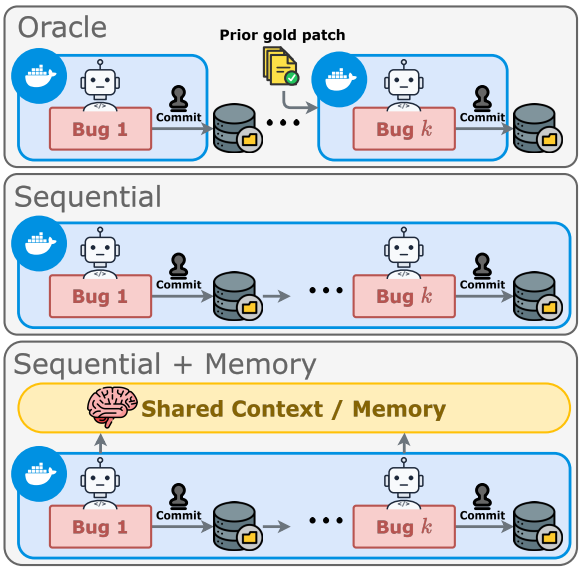


Figure 3: The three evaluation modes of ChainSWE: Oracle (oracle prior fixes), Seq (cumulative repository, fresh agent), and Seq+Mem (cumulative repository, persistent agent).

4.3 Evaluation Modes

ChainSWE scores each agent-model pair in three modes, as demonstrated in Figure 3, each exercising a distinct dimension of context management.

Oracle is the chain-aware and self-contained analogue of the standard SWE-bench protocol. For each bug in the chain, the container is reset to the base commit, then all earlier bugs in the chain have their gold test patches and gold fix patches applied before the agent starts. The agent never sees its own prior patches; instead, it operates on an ora-

cle codebase matching the original maintenance history.

SEQ (Sequential). The agent works through bugs 1 to n in order inside one container that is not reset between bugs. After each bug, the model-generated patch is committed to the repository, and the next bug starts from this updated codebase. The agent harness is restarted between bugs, so the conversation is fresh, but the repository carries the full history of model patches. A chain is not aborted at the first or second failed bug to ensure a fair comparison with ORACLE because downstream fixes may still fix the chain cleanly.

SEQ+MEM (Sequential + Memory). This mode is identical to SEQ except that the agent's conversation history is also preserved across bugs. The agent enters bug $k{+}1$ with the transcript from bug k, including intermediate observations and dead ends. This mode isolates the effect of *transcript-level memory* from *repository-level state*.

ORACLE provides an oracle-state per-bug baseline, independent of error propagation, while SEQ and SEQ+MEM measure the cost of carrying model-modified repository state across a chain. The gap between these modes quantifies how much performance is lost due to accumulated agent and repo state rather than genuine bug difficulty.

4.4 Experiment Settings

All experiments are executed on a CPU-only Linux server (x86_64) with 32 cores and 64 GB of RAM, running 4 evaluations in parallel. For all agents, we enforce a per-instance limit of 100 turns and a per-instance timeout of 30 minutes; all other parameters use the default configurations of each harness.

4.5 Metrics

We report **%Resolved**, which refers to the percentage of task instances where the agent-generated patches pass all tests. Because CHAINSWE evaluates ordered bug sequences, we also report per-chain success by position and full-chain success rate, where a chain is successful only if all bugs are resolved. Finally, we report average API cost per task and per chain, computed from the input and output tokens consumed during each rollout.

5 Results

5.1 Overall Success Rate

Because ORACLE already presents the agent with every earlier issue and its gold patch, the gap between ORACLE and the two sequential modes measures the cost of operating on imperfect, self-generated repository state. The remaining gap between SEQ and SEQ+MEM measures the effect of carrying the interaction history across bugs while still solving on self-generated state.

Sequential execution sharply reduces per-bug accuracy. Averaged over the seven models and three configurations, per-bug accuracy drops from 58.9% under ORACLE to 36.5% under SEQ and 36.9% under SEQ+MEM, a roughly 38% relative decline. Every one of the 21 model$\times$configuration cells loses accuracy when moving from ORACLE to SEQ, so the effect is not driven by a few weak models or settings. For SEQ, the decline ranges from 29% for `GPT-5.5` under BASELINE ($69.1 \rightarrow 49.0$) to 48% for `DeepSeek-V4-Pro` under SUB-AGENT ($48.4 \rightarrow 25.0$). Chain accuracy is less diagnostic because a single missed bug makes the whole chain fail; it is already low under ORACLE (mean 20.0%). Under SEQ the mean is comparable (17.0%), most model–configuration pairs fall, as later bugs inherit the state drift and errors left behind by earlier self-generated edits. A few actually improve, e.g. `GPT-5.5` and `Gemini-3.1-Pro` under SUMMARIZE. This is because a correct earlier fix leaves behind context that helps with the next bug.

Robustness to accumulated state varies across models. The relative ORACLE-to-SEQ degradation in per-bug accuracy, averaged over configurations, ranges from 36% for `GPT-5.4-mini` to 42% for `DeepSeek-V4-Pro`. The most robust models are those able to tell which of the previously accumulated edits are relevant to the current bug and which are not, and to reason about the effects those edits have already had on the repository. Under the raw-history baseline BASELINE, `GPT-5.5` is the most robust model (-29%), but its advantage erodes under the rewriting configurations (-38% under SUMMARIZE, -44% under SUB-AGENT). In comparison, smaller GPT models show more consistent and uniform degradation. Among the models with competitive ORACLE accuracy, `Claude-Opus-4.7` shows one of the largest average gaps (-39%), indicating that strong single-shot ability does not guarantee robustness to self-

Model	Mode	BASELINE			SUMMARIZE			SUB-AGENT		
		Per-Bug (%)	Chain (%)	Cost ($)	Per-Bug (%)	Chain (%)	Cost ($)	Per-Bug (%)	Chain (%)	Cost ($)
GPT-5.5	ORACLE	69.1	26.0	6.0	68.1	23.0	6.2	62.5	22.0	4.8
	SEQ	49.0 −29%	23.0 −12%	5.6	42.4 −38%	25.0 +9%	6.2	35.2 −44%	20.0 −9%	4.6
	SEQ+MEM	47.4 −31%	19.0 −27%	6.4	49.7 −27%	23.0 −0%	6.8	43.4 −31%	21.0 −5%	4.3
GPT-5.4-mini	ORACLE	61.8	22.0	1.1	61.8	26.0	1.1	36.2	10.0	1.2
	SEQ	41.8 −32%	15.0 −32%	1.1	39.8 −36%	21.0 −19%	1.2	21.7 −40%	10.0 −0%	1.3
	SEQ+MEM	41.8 −32%	17.0 −23%	1.4	37.8 −39%	18.0 −31%	1.2	22.0 −39%	9.0 −10%	1.1
GPT-5.4-nano	ORACLE	64.8	27.0	0.4	62.8	22.0	0.3	37.8	12.0	0.2
	SEQ	45.1 −30%	21.0 −22%	0.3	33.9 −46%	16.0 −27%	0.3	23.0 −39%	12.0 −0%	0.2
	SEQ+MEM	44.1 −32%	18.0 −33%	0.4	35.5 −43%	18.0 −18%	0.4	18.1 −52%	9.0 −25%	0.2
Claude-Opus-4.7	ORACLE	64.5	22.0	1.8	65.5	24.0	1.8	63.5	21.0	3.2
	SEQ	40.5 −37%	18.0 −18%	1.7	39.5 −40%	20.0 −17%	1.6	38.0 −40%	17.0 −19%	3.1
	SEQ+MEM	39.5 −39%	17.0 −23%	2.1	41.5 −37%	20.0 −17%	2.3	40.0 −37%	19.0 −10%	3.6
Claude-Opus-4.5	ORACLE	64.0	21.0	1.5	66.5	24.0	1.5	62.5	20.0	5.6
	SEQ	43.0 −33%	19.0 −10%	1.5	40.5 −39%	18.0 −25%	1.4	38.5 −38%	17.0 −15%	5.2
	SEQ+MEM	37.5 −41%	16.0 −24%	3.1	40.0 −40%	20.0 −17%	2.9	40.0 −36%	20.0 −0%	6.7
DeepSeek-V4-Pro	ORACLE	42.8	10.0	1.5	50.3	16.0	1.9	48.4	12.0	2.0
	SEQ	29.6 −31%	5.0 −50%	1.5	27.3 −46%	11.0 −31%	1.9	25.0 −48%	11.0 −8%	2.0
	SEQ+MEM	28.6 −33%	9.0 −10%	1.7	26.0 −48%	8.0 −50%	2.0	28.0 −42%	11.0 −8%	2.1
Gemini-3.1-Pro	ORACLE	61.8	20.0	1.3	61.5	19.0	1.5	61.5	20.0	1.9
	SEQ	36.5 −41%	18.0 −10%	1.4	39.5 −36%	20.0 +5%	1.4	37.2 −40%	19.0 −5%	1.9
	SEQ+MEM	37.5 −39%	18.0 −10%	1.9	39.8 −35%	21.0 +11%	1.9	37.2 −40%	19.0 −5%	2.5

Table 2: Main results on CHAINSWE with SWE-EDIT under three context-management configurations (BASELINE, SUMMARIZE, SUB-AGENT). We report all 100 chains (304 bugs); under SEQ and SEQ+MEM. **Per-Bug** is per-bug accuracy out of the 304 bugs; **Chain** is the fraction of the 100 chains for which all bugs pass; **Cost** is the estimated average USD per chain. Red subscripts on SEQ and SEQ+MEM rows give the relative change from the corresponding ORACLE baseline (green + marks an increase).

generated state.

`GPT-5.5` shows the largest observed gain from conversation memory. Carrying the conversation history across bugs (SEQ+MEM) does not improve the performance, as mean per-bug accuracy is essentially unchanged from SEQ (36.9% vs. 36.5%). `GPT-5.5` improves from a config-averaged 42.2% under SEQ to 46.8% under SEQ+MEM, with gains of +7.3 points under SUMMARIZE (42.4 → 49.7) and +8.2 points under SUB-AGENT (35.2 → 43.4). The other six models see no such benefit: SEQ+MEM stays within about two points of SEQ on average and is often slightly worse, with even `DeepSeek-V4-Pro` essentially unchanged (config-averaged 27.3 → 27.5). Preserved history compensates for imperfect repository state only when the model can productively reason over the longer history; otherwise, it adds input tokens without a measurable benefit.

Lighter context management is more robust than active rewriting. Holding the model fixed, the context-management configuration determines how quickly sequential execution degrades. Averaged over models, the relative ORACLE-to-SEQ drop grows from 33% under BASELINE to 40% under SUMMARIZE and 41% under SUB-AGENT. Simply retaining the raw history (BASELINE) is therefore more robust than summarizing it (SUMMARIZE) or routing edits through an LLM editor (SUB-AGENT). Even though the latter two are designed to reduce context pressure, they degrade overall performance due to potential information loss and inconsistency.

Cost. Per-chain cost is driven primarily by model tier rather than execution mode: it ranges from $0.2–$0.4 for `GPT-5.4-nano` to $4.3–$6.8 for `GPT-5.5`. Within a model, SEQ+MEM is often slightly more expensive than SEQ because it carries longer histories, but the added context rarely translates into higher accuracy. SUMMARIZE and SUB-AGENT can sometimes reduce cost for expensive main-model runs by shortening or moving token-heavy context out of the main trajectory, but Table 2 shows that this saving often comes with lower robustness.

Context	Mode	Pos. 1	Pos. 2	Pos. 3
BASELINE	ORACLE	58.6	64.3	67.0
	SEQ	57.1	39.3 −39%	27.7 −59%
	SEQ+MEM	56.5	38.0 −41%	28.2 −58%
SUMMARIZE	ORACLE	58.0	66.0	68.4
	SEQ	60.0	35.4 −46%	20.6 −70%
	SEQ+MEM	59.3	36.6 −45%	22.0 −68%
SUB-AGENT	ORACLE	49.5	58.1	59.2
	SEQ	50.3	29.9 −49%	17.5 −70%
	SEQ+MEM	50.4	31.3 −46%	19.2 −67%

Table 3: Per-position resolution rate (%) by context-management configuration, averaged across the seven evaluated models. Red subscripts report the relative drop from the corresponding ORACLE baseline.

5.2 Success Rate by Position

Tables 3 and 4 break per-bug resolution down by a bug's position in its chain, computed over the 97 length-3 chains; the 3 longer chains (two of length 4, one of length 5) are too few for stable per-position estimates and are omitted. Position resolves the ambiguity in the aggregate numbers: it separates whether later bugs are intrinsically harder from whether accumulated state makes them harder.

Under oracle resets, later bugs are not harder. In ORACLE, each bug begins from the correct prior state, so by construction position reflects only intrinsic task difficulty rather than accumulated error; resolution is therefore expected to be flat or even rising with position. Averaged over configurations, `GPT-5.5` resolves 61.2%, 70.4%, and 75.9% at positions 1 to 3 (rising with position), and the same shape holds across models (`Claude-Opus-4.7`: 60.0/68.0/72.0; `DeepSeek-V4-Pro`: 44.3/48.8/53.3). Later bugs in a chain are therefore not inherently more difficult.

Under accumulation, difficulty compounds with depth. In both SEQ and SEQ+MEM, an agent's success rate drops sharply and consistently along the chain. Under SEQ, `GPT-5.5` falls from 62.5% at position 1 to 40.5% and then 28.2%; `GPT-5.4-nano` similarly drops from 50.5% to 32.0% to 21.6%. For the first-bug success rate across all seven models, it remains close to the isolated ORACLE baseline (e.g., 62.5% vs. 61.2% for `GPT-5.5`), because the agent faces a clean slate with no accumulated changes. The entire penalty for sequential fixing is therefore concentrated at positions 2 and 3. As the chain gets deeper, the performance gap between isolated (ORACLE) and sequential (SEQ) fixing widens significantly—eventually reaching a 37.5-point drop for `GPT-5.4-nano` at position 3 (59.1% down to 21.6%).

Mode	Pos. 1	Pos. 2	Pos. 3
GPT-5.5			
ORACLE	61.2	70.4	75.9
SEQ	62.5	40.5 −42%	28.2 −63%
SEQ+MEM	61.9	44.7 −37%	37.8 −50%
GPT-5.4-mini			
ORACLE	51.2	58.1	56.4
SEQ	51.9	33.7 −42%	20.6 −63%
SEQ+MEM	51.5	31.6 −46%	21.3 −62%
GPT-5.4-nano			
ORACLE	50.2	61.9	59.1
SEQ	50.5	32.0 −48%	21.6 −63%
SEQ+MEM	48.8	30.9 −50%	21.0 −65%
Claude-Opus-4.7			
ORACLE	60.0	68.0	72.0
SEQ	60.0	38.0 −44%	24.0 −67%
SEQ+MEM	60.5	39.0 −43%	25.0 −65%
Claude-Opus-4.5			
ORACLE	60.5	68.5	71.0
SEQ	61.0	39.0 −43%	24.5 −65%
SEQ+MEM	60.0	38.0 −45%	23.0 −68%
DeepSeek-V4-Pro			
ORACLE	44.3	48.8	53.3
SEQ	44.7	24.7 −49%	15.1 −72%
SEQ+MEM	45.4	26.1 −46%	14.1 −74%
Gemini-3.1-Pro			
ORACLE	60.1	63.9	66.3
SEQ	59.8	36.1 −44%	19.6 −70%
SEQ+MEM	59.8	36.8 −42%	19.9 −70%

Table 4: Per-position resolution rate (%) by model, averaged across the three context-management configurations. Red subscripts report the relative drop from the corresponding ORACLE baseline.

The configuration ordering is stable across positions. Table 3 shows that the configurations are roughly the same at position 1 (50–60% across all modes) and separate only as state accumulates. At position 3 under SEQ, BASELINE retains 27.7% against 20.6% for SUMMARIZE and 17.5% for SUB-AGENT—a 59% relative drop from the ORACLE baseline for BASELINE versus 70% for the rewriting configurations. SEQ+MEM provides a small, inconsistent improvement at the last position (e.g., SUB-AGENT, 17.5 → 19.2) but does not change the ordering. This is consistent with the model-level finding: under accumulation, context rewriting increases errors more than carrying the context in its original form.

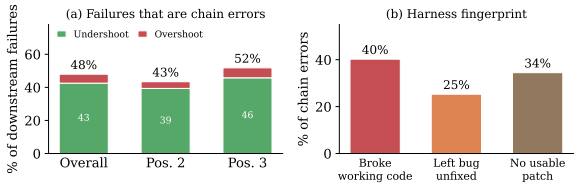


Figure 4: Chain-error analysis on BASELINE cells. **(a)** Of the downstream SEQ failures (positions 2–3), 48% overall (43% at position 2, 52% at position 3) are *chain errors*; each bar is split into the share of chain errors caused by an earlier *under-edit* (green) versus *over-edit* (red), roughly nine to one. **(b)** Harness fingerprint of the 318 chain errors that the agent attempted on the accumulated state.

5.3 Failure Mode Analysis

Since agents struggle increasingly with later bugs, we shift our focus to analyze failures at the chain level rather than the individual bug level, where the latter has already been thoroughly explored in prior works (Jimenez et al., 2024; Zhang et al., 2025). For every downstream bug (position ≥ 2), we re-tested it in isolation (ORACLE) starting from a *perfect* prior state. If a bug passes under ORACLE but fails under SEQ, the failure is clearly induced by the agent's accumulated errors, not the bug itself. We call such a failure a *chain error*. Across the BASELINE models, 318 of the 663 downstream SEQ failures at positions 2–3 (48%) are chain errors, and the rate grows with depth (43% at position 2 and 52% at position 3; Figure 4a): once state accumulates, roughly half of all later-bug failures are attributable to the chain rather than the bug itself.

As introduced in Section 1, depending on how an earlier bug's agent patch is miscalibrated *relative to that earlier bug's own gold patch*, chain errors divide into two symmetric families: **Overshoot** and **Undershoot**. These two errors aren't mutually exclusive—by the time an agent reaches the third bug, earlier patches have typically both over- and under-edited the codebase. However, when we can trace a chain error back to a single shared file, prior omissions (under-edits) clearly outnumber prior over-edits (Figure 4a). This mirrors our earlier finding: agents are much more likely to leave a later bug's preconditions unmet than to actively overwrite working code.

Regardless of the cause, the chain errors the agent attempts on the accumulated state show up in the testing harness in one of three distinct ways. As Figure 4b shows, the most common outcome is that the agent breaks previously working code simply because it is forced to build upon an already corrupted repository state:

- **Broke working code** (40%): the later bug's edit composes onto the corrupted state and a previously-passing PASS_TO_PASS test breaks.
- **Left the bug unfixed** (25%): the later bug's own FAIL_TO_PASS target cannot be made to pass because a precondition is missing, but nothing else breaks.
- **Produced no usable patch** (34%): the inherited state prevents the agent from producing or applying a usable patch at all, or leaves the suite uncollectable.

The fact that **broke working code** is the most common outcome—around 40% at both downstream positions—helps explain why preserving a richer chat history (SEQ+MEM) rarely helps. The structural damage is already baked into the repository state before the agent even begins tackling the later bug, meaning a longer conversation history simply cannot undo it.

6 Conclusion

We introduced CHAINSWE, the first benchmark that evaluates coding agents on sequential, dependent bug fixes within a shared codebase. By mining chains of real issues from six SWE-bench-family datasets and validating them through containerized test replay, CHAINSWE exposes failure modes that are not exposed when bugs are graded in isolation, with per-bug success dropping by up to 70% at the deepest chain positions. Over models and context-management configurations, we demonstrate that carrying the history of previous fixes yields only a marginal gain, whereas active summarization and sub-agent editing exhibit larger degradation. These results offer insights for future work on long-context, dependency-aware reasoning in code LMs and on repository-state management in agent harnesses.

Limitations

The six source benchmarks used to construct chain tasks inevitably contain noisy instances, where tests enforce implementation details or extra functionality not specified in the problem description (OpenAI, 2026). As a result, a small portion of chain

failures can stem not from agents incorrectly following earlier issue descriptions, but from the impossibility of inferring the hidden requirements of later tests from the problem descriptions of earlier issues.

The current release is restricted to Python and is modest in scale. Because long contiguous issue sequences are rare in existing benchmarks, we are unable to construct chain tasks longer than five instances from the current data pool. This limits both the generalizability of our findings to other programming languages and ecosystems and our ability to evaluate error accumulation and context management over longer maintenance horizons. Nonetheless, the construction pipeline itself is not specific to Python and can be extended as executable SWE-style datasets expand in language coverage, repository coverage, and size.

Acknowledgements

The views expressed in this paper are solely those of the authors and do not reflect those of NVIDIA, which does not sponsor or endorse this research.

References

Anthropic. 2025. Claude code. https://code.claude.com/.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *Preprint*, arXiv:2505.20411.

Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. 2026. Swe-rebench v2: Language-agnostic swe task collection at scale. *Preprint*, arXiv:2602.23866.

Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, Ruihua Song, Chang Liu, Cheng Chen, Kai Jia, and Ji-Rong Wen. 2026. Beyondswe: Can current code agent survive beyond single-repo bug fixing? *Preprint*, arXiv:2603.03194.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, and 3 others. 2025. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *Preprint*, arXiv:2509.16941.

Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, and 30 others. 2026. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *Preprint*, arXiv:2512.12730.

Ruida Hu, Chao Peng, Junjielong Xu, and Cuiyun Gao. 2026. Repo2run: Automated building executable environment for code repository at scale. *Advances in Neural Information Processing Systems*, 38:32679–32718.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *Preprint*, arXiv:2403.07974.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Kenan Li, Qirui Jin, Liao Zhu, Xiaosong Huang, Yijia Wu, Yikai Zhang, Xin Zhang, Zijian Jin, Yufan Huang, Elsie Nallipogu, Chaoyun Zhang, Yu Kang, Saravan Rajmohan, Qingwei Lin, Wenke Lee, and Dongmei Zhang. 2026a. Oracle-swe: Quantifying the contribution of oracle information signals on swe agents. *Preprint*, arXiv:2604.07789.

Kenan Li, Rongzhi Li, Linghao Zhang, Qirui Jin, Liao Zhu, Xiaosong Huang, Geng Zhang, Yikai Zhang, Shilin He, Chengxing Xie, Xin Zhang, Zijian Jin, Bowen Li, Chaoyun Zhang, Yu Kang, Yufan Huang, Elsie Nallipogu, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2026b. Repolaunch: Automating build and management of code repositories across languages and platforms. *Preprint*, arXiv:2603.05026.

Lifei Liu, Haoran Yu, Xiaochong Jiang, Su Wang, Pin Qian, and Yihang Chen. 2026. Operational reframing and approval-framed delegation in multi-agent llm safety. *arXiv preprint arXiv:2607.07097*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*.

Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. Agentboard: An analytical evaluation board of multi-turn llm agents.

Advances in neural information processing systems, 37:74325–74362.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870.

OpenAI. 2026. Why swe-bench verified no longer measures frontier coding capabilities. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/. Accessed: 2026-06-19.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2025. Training software engineering agents and verifiers with swe-gym. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*. ArXiv:2412.21139, accepted at ICML 2025.

Chong Peng, Pin Qian, Su Wang, Yihang Chen, and Varun Sah. 2026. Bap-sql: Budget-aware observation planning for agentic text-to-sql. *arXiv preprint arXiv:2608.02876*.

Shanghaoran Quan, Jiaxi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, Zekun Wang, Jian Yang, Zeyu Cui, Yang Fan, Yichang Zhang, Binyuan Hui, and Junyang Lin. 2025. Codeelo: Benchmarking competition-level code generation of llms with human-comparable elo ratings. *Preprint*, arXiv:2501.01257.

Su Wang, Pin Qian, Yihang Chen, Junxian You, Xiaoyuan Wang, Xiaochong Jiang, Lifei Liu, Haoran Yu, and Jingzhou Xu. 2026a. When safe skills collide: Measuring compositional risk in agent skill ecosystems. *arXiv preprint arXiv:2606.00448*.

Su Wang, Pin Qian, Yifan Lin, Jingzhou Xu, Yihang Chen, Xiaochong Jiang, Lifei Liu, and Haoran Yu. 2026b. Phantom guardrails: When self-improving agent harnesses fix failures that never happened. *arXiv preprint arXiv:2607.13083*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. 2026. Programbench: Can language models rebuild programs from scratch? *Preprint*, arXiv:2605.03546.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. Swe-bench goes live! *arXiv preprint arXiv:2505.23419*.

Yikai Zhang, Jiaxin Pei, Kenan Li, Qirui Jin, Maoquan Wang, Jin Pan, Yu Kang, Shengyu Fu, Elsie Nallipogu, Junjie Hu, Yufan Huang, and Zijian Jin. 2026. Swe-edit: Rethinking code editing for efficient swe-agent. *Preprint*, arXiv:2604.26102.

Jiale Zhao, Guoxin Chen, Fanzhe Meng, Minghao Li, Jie Chen, Hui Xu, Yongshuai Sun, Wayne Xin Zhao, Ruihua Song, Yuan Zhang, Peng Wang, Cheng Chen, Jirong Wen, and Kai Jia. 2026. Immersion in the github universe: Scaling coding agents to mastery. *Preprint*, arXiv:2602.09892.

A Dataset Details

A.1 Data Format

Each ChainSWE instance is a *chain*: a time-ordered list of bug-fix tasks over a single repository, sharing one Docker image and base commit. Table 5 lists the chain-level fields, and Table 6 the per-bug fields stored inside `bug_fixes`. Each per-bug entry is itself a self-contained, SWE-bench-style task instance.

A.2 Per-Repository Breakdown

ChainSWE spans 54 repositories with varying chain counts. Table 9 lists all 54 repositories with their chain and bug counts; `tobymao/sqlglot` (12 chains) and `s-knibbs/dataclasses-jsonschema` (7) are the largest.

Table 5: Chain-level fields of a CHAINSWE instance.

Field	Type	Description
`continuous_id`	`str`	Unique identifier for the chain.
`repo`	`str`	Source repository in `owner/name` form.
`base_commit`	`str`	Commit the chain starts from; the repository is reset here before the first bug.
`source`	`str`	Originating SWE-bench-family dataset.
`date_range`	`str`	Span of the earliest to latest bug date in the chain.
`docker_image`	`str`	Pre-built execution environment shared by every bug in the chain.
`swebench_instance_ids`	`List[str]`	Ordered list of the underlying SWE-bench instance IDs in the chain.
`bug_fixes`	`List[dict]`	The ordered bug-fix tasks; each entry has the fields in Table 6.

Table 6: Per-bug fields stored in each entry of `bug_fixes`.

Field	Type	Description
`order`	`int`	Position of the bug within the chain (1-indexed).
`swebench_instance_id`	`str`	Identifier of the underlying SWE-bench task instance.
`commit_hash`	`str`	Commit at which this bug's gold patch was merged.
`problem_statement`	`str`	Issue description used as the task prompt for this bug.
`date`	`str`	Date of the bug's fixing commit, used to order the chain.
`source`	`str`	Originating SWE-bench-family dataset for this bug.
`patch`	`str`	Gold patch proposed by the pull request.
`test_patch`	`str`	Modifications to the test suite to check whether the issue is resolved.
`FAIL_TO_PASS`	`List[str]`	Tests expected to change from failing to passing once fixed.
`PASS_TO_PASS`	`List[str]`	Tests already passing before the fix.
`test_cmds`	`str`	Command(s) used to run the test suite.

Table 7: Per-Source breakdown for CHAINSWE.

Source	Chains
SWE-rebench-v2	52
SWE-rebench	17
SWE-bench Live	12
SWE-Gym	7
SWE-bench	6
Mixed	6
SWE-bench Pro	0

Table 8: Average maximum context reached per chain, averaged across all models.

Config	ORACLE	SEQ	SEQ+MEM
BASELINE	48.2k	48.1k	63.6k
SUMMARIZE	42.7k	41.8k	48.4k
SUB-AGENT	42.8k	42.6k	53.0k

A.3 Per-Source Breakdown

According to Table 7, SWE-rebench-v2 contributes the largest share with 52 chains (52%), followed by SWE-rebench (17 chains), SWE-bench Live (12), SWE-Gym (7), SWE-bench (6), and chains spanning multiple sources (6); SWE-bench Pro contributes no chains that survive filtering.

A.4 Context Usage

To quantify how much context each configuration actually consumes, we record the peak input-context size (in tokens) reached during every agent run, take the maximum over the bugs in a chain, and average across all models and chains. Table 8 reports this average maximum context for each context-management configuration (BASELINE, SUMMARIZE, SUB-AGENT) under the three evaluation modes (ORACLE, SEQ, SEQ+MEM). SEQ+MEM consistently reaches the largest context—10–16k tokens above the corresponding ORACLE and SEQ runs—because it carries the agent's conversation history across bugs. This confirms that CHAINSWE exercises markedly longer context than common SWE-related tasks.

B Contamination Analysis

Performance by source dataset. Table 10 reports per-bug success rates grouped by source dataset and averaged across all evaluated language models and agent configurations. Both SEQ and SEQ+MEM underperform ORACLE in every source group. The difference between ORACLE and the sequential modes ranges from 2.0 to 10.8 percentage. This indicates that the degradation under accumulated agent-generated state is not limited to a single

Table 9: Per-repository breakdown of CHAINSWE (54 repositories, 100 chains, 304 bugs), sorted by chain count.

Repository	Chains	Bugs	Repository	Chains	Bugs
tobymao/sqlglot	12	36	drivendataorg/cloudpathlib	1	3
s-knibbs/dataclasses-jsonschema	7	21	falconry/falcon	1	4
Project-MONAI/MONAI	4	12	fonttools/fonttools	1	3
brazilian-utils/brutils-python	4	12	hdmf-dev/hdmf	1	3
pyccel/pyccel	4	12	hikari-py/hikari	1	3
reata/sqllineage	4	12	instructlab/instructlab	1	3
HypothesisWorks/hypothesis	3	9	ipython/ipython	1	3
MartinThoma/flake8-simplify	3	9	jazzband/tablib	1	3
PyCQA/docformatter	3	9	joke2k/faker	1	3
amaranth-lang/amaranth	3	9	jsonpickle/jsonpickle	1	3
pydata/xarray	3	9	keras-team/keras	1	3
sdv-dev/RDT	3	9	koxudaxi/datamodel-code-generator	1	3
getmoto/moto	2	6	matthewwithanm/python-markdownify	1	3
litestar-org/polyfactory	2	6	meltano/sdk	1	3
mikedh/trimesh	2	7	oscal-compass/compliance-trestle	1	3
modin-project/modin	2	6	pallets/click	1	3
mwaskom/seaborn	2	6	pydantic/pydantic	1	3
Kozea/WeasyPrint	1	3	pydicom/pydicom	1	3
NeurodataWithoutBorders/pynwb	1	3	pytask-dev/pytask	1	3
Python-Markdown/markdown	1	3	pytest-dev/pyfakefs	1	3
Textualize/rich	1	3	reanahub/reana-client	1	3
aio-libs/aiohttp	1	3	relekang/python-semantic-release	1	3
alteryx/woodwork	1	3	samuelcolvin/pydantic	1	3
beeware/briefcase	1	5	sanic-org/sanic-ext	1	3
codezonediitj/pydatastructs	1	3	sissbruecker/linkding	1	3
copier-org/copier	1	3	spulec/freezegun	1	3
deepset-ai/haystack	1	3	stanfordnlp/dspy	1	3

Table 10: Per-bug success rate (%) by source dataset, averaged across all evaluated language models and agent configurations.

Source	ORACLE	SEQ	SEQ+MEM
Mixed	44.3	40.5	37.6
SWE-Gym	59.2	52.9	54.9
SWE-rebench	74.4	68.2	67.6
SWE-rebench-v2	56.0	46.1	45.2
SWE-bench	57.4	55.4	54.9
SWE-bench Live	41.4	37.2	36.7

source dataset.

Performance by issue date relative to model release. Absolute issue dates do not account for differences among model release dates. We therefore additionally group each evaluation according to the time between the issue date and the release of the evaluated model. As shown in Table 11, SEQ and SEQ+MEM remain below ORACLE in all four temporal groups. This includes issues dated within one year before model release, for which success decreases from 46.7% under ORACLE to 42.4% under both sequential modes. The degradation therefore does not occur only on issues long before the models were released.

Table 11: Per-bug success rate (%) by the time between issue date and model release, averaged across all evaluated language models and agent configurations.

Gap to model release	ORACLE	SEQ	SEQ+MEM
$\leq$ 1 year before release	46.7	42.4	42.4
1–2 years before release	53.2	46.0	44.2
2–4 years before release	56.5	48.7	48.6
$>$ 4 years before release	65.9	59.0	58.5

C Failure Mode Examples

We give a concrete, verified instance of each case in the chain-error taxonomy of Section 5.3. Every example is a real BASELINE rollout. In each one, the downstream bug we point to is solved correctly when the agent starts from the oracle prior state (ORACLE), but fails when it starts from the state the agent itself accumulated (SEQ)—which is exactly what we mean by a chain error.

Cause: overshoot. In `stanfordnlp/dspy` (`GPT-5.5`), the agent overshoots while fixing bug 2 (`dspy-1609`). The intended fix only touches `chat_adapter.py`, `predict.py`, `signatures/utils.py`, and `utils/dummies.py`, but the agent also modifies `functional/functional.py`, a file outside the

gold patch. This stray change later interferes with bug 3 (dspy-1651), which also edits functional/functional.py. Bug 3 is solved correctly from the oracle state (ORACLE), yet in SEQ its patch is applied on top of the earlier unintended modification, causing the final repository state to diverge from the oracle. As a result, four previously passing test_signature_typed tests begin to fail. The agent's full patch for bug 2 is shown below; note the second hunk, on functional.py, which lies outside the gold set.

```
diff --git a/dspy/adapters/chat_adapter.py b/dspy/
↪ adapters/chat_adapter.py
index 5a20dcff..8f8213ed 100644
--- a/dspy/adapters/chat_adapter.py
+++ b/dspy/adapters/chat_adapter.py
@@ -85,8 +85,15 @@ def format_list(items):
     return "\n".join([f"[{idx+1}] {format_blob(txt)
↪ }" for idx, txt in enumerate(items)])


-def _format_field_value(value) -> str:
-    if isinstance(value, list):
+def _format_field_value(value, field_info=None) ->
↪ str:
+    if field_info is not None:
+        formatter = (field_info.json_schema_extra or
↪  {}).get("format")
+        if formatter is not None:
+            value = formatter(value)
+
+    if isinstance(value, bytes):
+        return value.decode("utf-8")
+    elif isinstance(value, list):
         return format_list(value)
     elif isinstance(value, pydantic.BaseModel):
         return value.model_dump_json()
@@ -94,10 +101,11 @@ def _format_field_value(value)
↪ -> str:
         return str(value)


-def format_fields(fields):
+def format_fields(fields, field_infos=None):
     output = []
     for k, v in fields.items():
-        v = _format_field_value(v)
+        field_info = field_infos.get(k) if
↪ field_infos is not None else None
+        v = _format_field_value(v, field_info)
         output.append(f"[[ ## {k} ## ]]\n{v}")

     return "\n\n".join(output).strip()
@@ -132,7 +140,12 @@ def format_turn(signature,
↪ values, role, incomplete=False):
         if not set(values).issuperset(set(
↪ field_names)):
             raise ValueError(f"Expected {field_names}
↪  but got {values.keys()}")

-    content.append(format_fields({k: values.get(k, "
↪ Not supplied for this particular example.") for k
↪ in field_names}))
+    content.append(
+        format_fields(
+            {k: values.get(k, "Not supplied for this
↪  particular example.") for k in field_names},
+            signature.fields,
+        ),
+    )

     if role == "user":
         content.append(
diff --git a/dspy/functional/functional.py b/dspy/
↪ functional/functional.py
index a45e302c..598e6658 100644
--- a/dspy/functional/functional.py
+++ b/dspy/functional/functional.py
@@ -240,7 +240,7 @@ class TypedPredictor(dspy.Module):
↪
                         schema = json.dumps(type_.
↪ model_json_schema())
                     else:
                         adapter = pydantic.
↪ TypeAdapter(type_)
-                        to_json = lambda x: adapter.
↪ serializer.to_json(x)
+                        to_json = lambda x: adapter.
↪ serializer.to_json(x).decode("utf-8")
                         from_json = lambda x, type_=
↪ adapter: type_.validate_json(x)
                         schema = json.dumps(adapter.
↪ json_schema())
                     if self.wrap_json:
```

Cause: undershoot. In s-knibbs/dataclasses-jsonschema (DeepSeek-V4-Pro), all three bugs involve the same file, dataclasses_jsonschema/__init__.py. When fixing bug 2 (dataclasses-jsonschema-112), the agent fails to modify the target file and instead generates a standalone script that merely reproduces the issue and bypass the issue. As a result, the repository state after bug 2 remains incomplete, with the required changes to __init__.py never applied. Bug 3 (dataclasses-jsonschema-116) is solved correctly from the oracle state (ORACLE), but under SEQ its patch is applied to a repository that is still missing bug 2's fix, causing the final state to diverge from the oracle and several previously passing tests to fail. Agent's entire patch for bug 2 consists solely of the following throwaway script.

```
diff --git a/reproduce_subclass_override.py b/
↪ reproduce_subclass_override.py
new file mode 100644
index 0000000..17834ea
--- /dev/null
+++ b/reproduce_subclass_override.py
@@ -0,0 +1,59 @@
+import sys
+sys.path.insert(0, '/workspace/dataclasses-
↪ jsonschema/')
+
+from typing import Literal
+from dataclasses import dataclass
+
+from dataclasses_jsonschema import JsonSchemaMixin
+
+
+@dataclass
+class BaseObject(JsonSchemaMixin):
+    field: str
+
+
+@dataclass
+class NarrowedObject(BaseObject, JsonSchemaMixin):
+    field: Literal['staticstr']
+
+
+print("Testing BaseObject:")
+print(BaseObject.json_schema())
+print()
+
```

```
+print("Testing NarrowedObject:")
+try:
+    schema = NarrowedObject.json_schema()
+    print(schema)
+except Exception as e:
+    print(f"Error: {type(e).__name__}: {e}")
+
+print()
+
+# Also test with embeddable=True
+print("Testing NarrowedObject embeddable:")
+try:
+    schema = NarrowedObject.json_schema(embeddable=
↪ True)
+    print(schema)
+except Exception as e:
+    print(f"Error: {type(e).__name__}: {e}")
+
+print()
+
+# Test deserialization
+print("Test from_dict:")
+try:
+    obj = NarrowedObject.from_dict({"field": "
↪ staticstr"})
+    print(f"  Success: {obj}")
+except Exception as e:
+    print(f"  Error: {type(e).__name__}: {e}")
+
+print()
+
+# Test deserialization with wrong value
+print("Test from_dict with wrong value:")
+try:
+    obj = NarrowedObject.from_dict({"field": "
↪ wrongvalue"})
+    print(f"  Success: {obj}")
+except Exception as e:
+    print(f"  Error: {type(e).__name__}: {e}")
```

Fingerprint: broke working code. In `meltano/sdk` (GPT-5.5), bug 2 (`meltano-1880`) is resolved correctly under ORACLE. However, in SEQ, although the agent successfully makes the target bug pass, it applies the same modification to `singer_sdk/target_base.py` on top of an inherited state where earlier behavior has not been fully accounted for. This results in a regression test, `test_add_sqlsink_and_get_sink`, which was previously passing, begins to fail again. Thus, while the current bug is fixed, previously correct functionality is inadvertently broken. The full agent patch is shown below.

```
diff --git a/singer_sdk/target_base.py b/singer_sdk/
↪ target_base.py
index fe2bc04..0d79d4c 100644
--- a/singer_sdk/target_base.py
+++ b/singer_sdk/target_base.py
@@ -12,12 +12,14 @@ import typing as t
 import click
 from joblib import Parallel, delayed,
↪ parallel_backend

+from singer_sdk.configuration._dict_config import
↪ merge_missing_config_jsonschema
 from singer_sdk.connectors import SQLConnector
 from singer_sdk.exceptions import
↪ RecordsWithoutSchemaException
 from singer_sdk.helpers._batch import
↪ BaseBatchFileEncoding
 from singer_sdk.helpers._classproperty import
↪ classproperty
 from singer_sdk.helpers._compat import final
 from singer_sdk.helpers.capabilities import (
+    BATCH_CONFIG,
     TARGET_SCHEMA_CONFIG,
     CapabilitiesEnum,
     PluginCapabilities,
@@ -104,6 +106,23 @@ class Target(PluginBase,
↪ SingerReader, metaclass=abc.ABCMeta):
             PluginCapabilities.FLATTENING,
         ]

+    @classmethod
+    def append_builtin_config(cls: type[Target],
↪ config_jsonschema: dict) -> None:
+        """Appends built-in config to `
↪ config_jsonschema` if not already set.
+
+        To customize or disable this behavior,
↪ developers may either override this class
+        method or override the `capabilities`
↪ property to disable any unwanted
+        built-in capabilities.
+
+        Args:
+            config_jsonschema: Config JSON schema to
↪  append built-in config to.
+        """
+        super().append_builtin_config(
↪ config_jsonschema)
+
+        capabilities = cls.capabilities
+        if PluginCapabilities.BATCH in capabilities:
+            merge_missing_config_jsonschema(
↪ BATCH_CONFIG, config_jsonschema)
+
     @property
     def max_parallelism(self) -> int:
         """Get max parallel sinks.
```

Fingerprint: left the bug unfixed. In `amaranth-lang/amaranth` (GPT-5.5), bug 1 (`amaranth-1265`) is left half-finished. Its gold fix spans four files `_ast.py`, `_ir.py`, `_xfrm.py`, and `sim/_pyrtl.py`, but the agent edits only `_ir.py` and `_xfrm.py` (together with the `__init__.py` and `ir.py` export lists), leaving `_ast.py` and `sim/_pyrtl.py` untouched. Bug 2 (`amaranth-1268`) then correctly updates its own target file, `lib/wiring.py`, without introducing additional regressions, but `test_is_compliant` still fails under SEQ because it runs on top of the incompletely fixed state from bug 1. The full patch for bug 1 is shown below.

```
diff --git a/amaranth/hdl/__init__.py b/amaranth/hdl/
↪ __init__.py
index 43852da..b967d73 100644
--- a/amaranth/hdl/__init__.py
+++ b/amaranth/hdl/__init__.py
@@ -6,8 +6,8 @@ from ._ast import Format, Print,
↪ Assert, Assume, Cover
 from ._ast import IOValue, IOPort
 from ._dsl import Module
 from ._cd import DomainError, ClockDomain
-from ._ir import UnusedElaboratable, Elaboratable,
↪ DriverConflict, Fragment
-from ._ir import Instance, IOBufferInstance
+from ._ir import UnusedElaboratable, Elaboratable,
↪ DuplicateElaboratable
+from ._ir import DriverConflict, Fragment, Instance,
↪  IOBufferInstance
 from ._mem import MemoryData, MemoryInstance, Memory,
↪  ReadPort, WritePort, DummyPort
 from ._rec import Record
```

```
 from ._xfrm import DomainRenamer, ResetInserter,
↪ EnableInserter
@@ -26,7 +26,7 @@ __all__ = [
     # _cd
     "DomainError", "ClockDomain",
     # _ir
-    "UnusedElaboratable", "Elaboratable", "
↪ DriverConflict", "Fragment",
+    "UnusedElaboratable", "Elaboratable", "
↪ DuplicateElaboratable", "DriverConflict", "
↪ Fragment",
     "Instance", "IOBufferInstance",
     # _mem
     "MemoryData", "MemoryInstance", "Memory", "
↪ ReadPort", "WritePort", "DummyPort",
diff --git a/amaranth/hdl/_ir.py b/amaranth/hdl/_ir.
↪ py
index 717db2e..7e7890c 100644
--- a/amaranth/hdl/_ir.py
+++ b/amaranth/hdl/_ir.py
@@ -9,8 +9,8 @@ from . import _ast, _cd, _ir, _nir


 __all__ = [
-    "UnusedElaboratable", "Elaboratable", "
↪ DriverConflict", "Fragment", "Instance",
-    "IOBufferInstance", "PortDirection", "Design", "
↪ build_netlist",
+    "UnusedElaboratable", "Elaboratable", "
↪ DuplicateElaboratable", "DriverConflict",
+    "Fragment", "Instance", "IOBufferInstance", "
↪ PortDirection", "Design", "build_netlist",
 ]


@@ -26,37 +26,50 @@ class Elaboratable(_unused.
↪ MustUse):
     _MustUse__warning = UnusedElaboratable


+class DuplicateElaboratable(UserWarning):
+    pass
+
+
 class DriverConflict(UserWarning):
     pass


 class Fragment:
+    _elaborating = 0
+
     @staticmethod
     def get(obj, platform):
         code = None
         origins = []
-        while True:
-            if isinstance(obj, Fragment):
-                if hasattr(obj, "origins"):
-                    obj.origins = tuple(origins)
-                return obj
-            elif isinstance(obj, Elaboratable):
-                code = obj.elaborate.__code__
-                UnusedElaboratable._MustUse__silence
↪  = False
-                obj._MustUse__used = True
-                new_obj = obj.elaborate(platform)
-            else:
-                raise TypeError(f"Object {obj!r} is
↪ not an 'Elaboratable' nor 'Fragment'")
-            if new_obj is obj:
-                raise RecursionError(f"Object {obj!r}
↪  elaborates to itself")
-            if new_obj is None and code is not None:
-                warnings.warn_explicit(
-                    message=".elaborate() returned
↪ None; missing return statement?",
-                    category=UserWarning,
-                    filename=code.co_filename,
-                    lineno=code.co_firstlineno)
-            origins.append(obj)
-            obj = new_obj
+        outermost = Fragment._elaborating == 0
+        Fragment._elaborating += 1
+        try:
+            while True:
+                if isinstance(obj, Fragment):
+                    if hasattr(obj, "origins"):
+                        obj.origins = tuple(origins)
+                    if outermost:
+                        obj.
↪ _check_duplicate_elaboratables()
+                    return obj
+                elif isinstance(obj, Elaboratable):
+                    code = obj.elaborate.__code__
+                    UnusedElaboratable.
↪ _MustUse__silence = False
+                    obj._MustUse__used = True
+                    new_obj = obj.elaborate(platform
↪ )
+                else:
+                    raise TypeError(f"Object {obj!r}
↪  is not an 'Elaboratable' nor 'Fragment'")
+                if new_obj is obj:
+                    raise RecursionError(f"Object {
↪ obj!r} elaborates to itself")
+                if new_obj is None and code is not
↪ None:
+                    warnings.warn_explicit(
+                        message=".elaborate()
↪ returned None; missing return statement?",
+                        category=UserWarning,
+                        filename=code.co_filename,
+                        lineno=code.co_firstlineno)
+                origins.append(obj)
+                obj = new_obj
+        finally:
+            Fragment._elaborating -= 1

     def __init__(self, *, src_loc=None):
         self.drivers = OrderedDict()
@@ -110,6 +123,26 @@ class Fragment:
         assert isinstance(subfragment, Fragment)
         self.subfragments.append((subfragment, name,
↪  src_loc))

+    def _check_duplicate_elaboratables(self):
+        origin_paths = OrderedDict()
+
+        def traverse(fragment, hierarchy):
+            for origin in getattr(fragment, "origins
↪ ", ()) or ():
+                origin_paths.setdefault(id(origin),
↪ (origin, []))[1].append(hierarchy)
+            for index, (subfragment, name, _src_loc)
↪  in enumerate(fragment.subfragments):
+                if name is None:
+                    name = f"<unnamed #{index}>"
+                traverse(subfragment, (*hierarchy,
↪ name))
+
+        traverse(self, ("top",))
+        for origin, paths in origin_paths.values():
+            if len(paths) <= 1:
+                continue
+            warnings.warn(
+                "Elaboratable {!r} is added to the
↪ hierarchy more than once: {}"
+                .format(origin, ", ".join(".".join(
↪ path) for path in paths)),
+                DuplicateElaboratable, stacklevel=3)
+
     def find_subfragment(self, name_or_index):
         if isinstance(name_or_index, int):
             if name_or_index < len(self.subfragments
↪ ):
diff --git a/amaranth/hdl/_xfrm.py b/amaranth/hdl/
↪ _xfrm.py
index 01b4fa1..e74176d 100644
--- a/amaranth/hdl/_xfrm.py
+++ b/amaranth/hdl/_xfrm.py
@@ -318,6 +318,8 @@ class FragmentTransformer:
                     )
         else:
             new_fragment = Fragment(src_loc=fragment.
↪ src_loc)
+        if hasattr(fragment, "origins"):
+            new_fragment.origins = fragment.origins
         new_fragment.attrs = OrderedDict(fragment.
↪ attrs)
```

```
        self.map_subfragments(fragment, new_fragment
↪ )
        self.map_domains(fragment, new_fragment)
diff --git a/amaranth/hdl/ir.py b/amaranth/hdl/ir.py
index db44e53..737c749 100644
--- a/amaranth/hdl/ir.py
+++ b/amaranth/hdl/ir.py
@@ -4,7 +4,10 @@ from .. import hdl as __hdl
 from . import _ir as __origin


-__all__ = ["UnusedElaboratable", "Elaboratable", "
↪ DriverConflict", "Fragment", "Instance"]
+__all__ = [
+    "UnusedElaboratable", "Elaboratable", "
↪ DuplicateElaboratable", "DriverConflict",
+    "Fragment", "Instance",
+]


 def __getattr__(name):
```

Fingerprint: produced no usable patch. In `brazilian-utils/brutils-python` (GPT-5.5), bug 2 (`brutils-216`) only partially applies the intended changes, modifying just two of the seven gold files `__init__.py` and `pis.py`, and primarily wiring up an import while leaving the rest of the package unchanged. Bug 3 (`brutils-218`) still succeeds under ORACLE, but in SEQ the partially updated state from bug 2 causes the test suite to fail at collection time (exit code 2). The agent's full patch for bug 2 is shown below.

```
diff --git a/brutils/__init__.py b/brutils/__init__.
↪ py
index e005bdb..e0b89c2 100644
--- a/brutils/__init__.py
+++ b/brutils/__init__.py
@@ -16,3 +16,4 @@ from brutils.cep import (
     remove_symbols as remove_symbols_cep,
     generate as generate_cep,
 )
+from brutils.pis import is_valid as is_valid_pis
diff --git a/brutils/pis.py b/brutils/pis.py
new file mode 100644
index 0000000..e1d1a5c
--- /dev/null
+++ b/brutils/pis.py
@@ -0,0 +1,48 @@
+# OPERATIONS
+#############
+
+
+def validate(pis):  # type: (str) -> bool
+    """
+    Returns whether or not the verifying checksum
↪ digit of the
+    given `pis` matches its base number. Input
↪ should be a digit
+    string of proper length.
+
+    Source: https://www.macoratti.net/alg_pis.htm
+    """
+
+    if not pis.isdigit() or len(pis) != 11 or len(
↪ set(pis)) == 1:
+        return False
+
+    return _hashdigit(pis) == int(pis[-1])
+
+
+def is_valid(pis):  # type: (str) -> bool
+    """
+    Evaluates that pis is String and calls validate.
+    """
+
+    return isinstance(pis, str) and validate(pis)
+
+
+def _hashdigit(pis):  # type: (str) -> int
+    """
+    Will compute the PIS checksum digit for the
↪ given `pis` input.
+    The input needs to contain the 10 base digits
↪ else computation
+    will yield the wrong result.
+    """
+
+    weights = (3, 2, 9, 8, 7, 6, 5, 4, 3, 2)
+    val = sum(int(digit) * weight for digit, weight
↪ in zip(pis[:10], weights)) % 11
+    digit = 11 - val
+
+    return 0 if digit in (10, 11) else digit
+
+
+def _checksum(basenum):  # type: (str) -> str
+    """
+    Will compute the checksum digit for a given PIS
↪ base number.
+    `basenum` needs to be a digit-string of adequate
↪  length.
+    """
+
+    return str(_hashdigit(basenum))
```

D Agent Prompts

This section lists the prompts used by the SWE-EDIT.

D.1 Viewer Subagent Prompt

When the `view` command is invoked under SUB-AGENT, the viewer subagent receives the numbered file contents and a natural-language query, and returns the line ranges relevant to that query so that only those ranges are read into the main trajectory. The complete system prompt is shown below.

Viewer Subagent System Prompt

```
You are an expert code analyzer. Your task
is to identify line ranges in a file that
are relevant to a given query.

You will be given:
1. A file with numbered lines in the format:
   LINE_NUMBER\tLINE_CONTENT
2. A query describing what the user is
   looking for

Your job is to analyze the file and return
the line ranges that are most relevant to
the query. Consider:
- Function/method definitions that match
  the query
- Class definitions related to the query
- Variable declarations or assignments
  relevant to the query
- Import statements if they're relevant
- Comments that explain relevant code
- Any code blocks that implement
  functionality related to the query

OUTPUT FORMAT:
You must output your response as a JSON
array of line ranges. Each range is an
array of two integers [start_line, end_line]
(inclusive, 1-indexed).
```

```
Example output:
[[10, 25], [45, 60], [100, 115]]

RULES:
1. Only output the JSON array, no
   additional explanation or comments
2. Line numbers are 1-indexed (first line
   is line 1)
3. Each range should include complete
   logical blocks (don't cut functions/
   classes in the middle)
4. Include a few lines of context before
   and after each relevant section when
   appropriate
5. If nothing in the file is relevant to
   the query, return an empty array: []
6. Ranges should be sorted by start line
   number
7. Merge overlapping or adjacent ranges
8. Keep ranges focused - don't include
   entire files unless the query asks for
   everything

Example 1 - Finding a specific function:
Query: "Where is the calculate_total
function defined?"
Output: [[15, 28]]

Example 2 - Finding multiple related
sections:
Query: "How is user authentication
handled?"
Output: [[5, 8], [23, 45], [102, 130]]

Example 3 - Nothing relevant found:
Query: "Where is the database connection
configured?"
Output: []

Now, analyze the file content and query
provided, and output the relevant line
ranges as a JSON array.
```

D.2 Editor Subagent Prompt

When the `edit` command is invoked, the editor subagent receives the file contents and the edit instruction. The complete system prompt is shown below.

Editor Subagent System Prompt

```
You are an expert code editor. Your task
is to analyze a file and make
modifications according to the provided
instructions.

You must output your changes using the
search-replace format shown below. You
can make multiple edits by including
multiple search-replace blocks.

Format for each edit:
<<<<<<< SEARCH
exact lines from the original file to find
=======
new lines to replace them with
>>>>>>> REPLACE

IMPORTANT RULES:
1. The SEARCH block must match the
   original file content EXACTLY,
   including whitespace and indentation
2. You can make multiple edits by
   including multiple search-replace
   blocks
3. If the SEARCH block is empty (no
   content between <<<<<<< SEARCH and
   =======), it means you want to REWRITE
   THE ENTIRE FILE with the content in
   the REPLACE block
4. Each SEARCH block must be unique in
   the file - if there are multiple
   matches, include more context
5. Only output the search-replace blocks,
   no additional explanation or comments

Example 1 - Modifying specific lines:
<<<<<<< SEARCH
def calculate_total(items):
    return sum(items)
=======
def calculate_total(items):
    if not items:
        return 0
    return sum(items)
>>>>>>> REPLACE

Example 2 - Multiple edits:
<<<<<<< SEARCH
import os
=======
import os
import sys
>>>>>>> REPLACE

<<<<<<< SEARCH
def main():
    pass
=======
def main():
    print("Hello, World!")
>>>>>>> REPLACE

Example 3 - Rewriting entire file
(empty SEARCH block):
<<<<<<< SEARCH
=======
#!/usr/bin/env python3
# New file content here
def new_function():
    pass
>>>>>>> REPLACE

Now, analyze the file content and
instruction provided, and output the
necessary search-replace blocks.
```

D.3 Context Summarization Prompt

Under the SUMMARIZE configuration, once the running context exceeds a token budget the earlier portion of the trajectory is replaced by a model-written summary. The summarizer is driven by the system prompt below.

Summarization System Prompt

```
You are a meticulous context-summarization
assistant for an autonomous
software-engineering agent. The agent is
solving a coding task and its conversation
has grown too long to keep in full. Your
job is to compress the earlier part of the
conversation into a dense, factual summary
that lets the agent continue working
without re-reading the original messages.

Preserve everything that is operationally
important and discard chatter. Produce a
structured summary with these sections
(omit a section only if it is genuinely
empty):
1. TASK: the problem statement / goal the
   agent is working toward.
2. REPO STATE: relevant files, directories,
   and code locations discovered (with
   paths and key symbols/line references
   when known).
```

```
3. CHANGES MADE: edits already applied to
   the codebase, described precisely enough
   to reconstruct what was done.
4. COMMANDS & RESULTS: important shell/test
   commands run and their outcomes
   (failures, tracebacks, test pass/fail).
5. CURRENT STATE & NEXT STEPS: where things
   stand and what remains to be done.
6. KEY FACTS: any other constraints,
   hypotheses, or observations worth
   keeping.

Be specific and concrete -- keep exact file
paths, function names, error messages, and
command strings. Do not invent information.
Do not ask questions or address the user;
output only the summary.
```

Summarization User Template

```
Summarize the following earlier portion of
the agent's conversation so the agent can
continue the task. This transcript is the
prefix that will be replaced by your
summary; the most recent messages are kept
verbatim and are NOT shown here.

<transcript>
{transcript}
</transcript>
```

D.4 System Prompt

SWE-Bench System Prompt

```
<uploaded_files>
{{ instance.repo_path }}
</uploaded_files>
I've uploaded a python code repository in
the directory {{ instance.repo_path }}
(not in /tmp/inputs). Consider the
following issue descriptions:

<issue_description>
{{ instance.problem_statement }}
</issue_description>

Can you help me implement the necessary
changes to the repository so that the
requirements specified in the
<issue_description> are met?
I've already taken care of all changes to
any of the test files described in the
<issue_description>. This means you DON'T
have to modify the testing logic or any
of the tests in any way!
Also the development Python environment
is already set up for you (i.e., all
dependencies already installed), so you
don't need to install other packages.

Your task is to make the minimal changes
to non-test files in the
{{ instance.repo_path }} directory to
ensure the <issue_description> is
satisfied.

Follow these steps to resolve the issue:
1. As a first step, it might be a good
   idea to explore the repo to
   familiarize yourself with its
   structure.
2. Create a script to reproduce the error
   and execute it with
   `python <filename.py>` using the
   execute_bash tool to confirm the error
   - **Important:** If testing a Python
     package, add
     `import sys; sys.path.insert(0,
     '{{ instance.repo_path }}')`
     at the top of your script before
     package imports to ensure you're
     testing the local version, not an
     installed version.
3. Edit the source code of the repo to
   resolve the issue
4. Rerun your reproduce script and
   confirm that the error is fixed!
5. Think about edge cases and make sure
   your fix handles them as well

Your thinking should be thorough and so
it's fine if it's very long.
```